Clase 10: Programación con memoria compartida OpenMP

IIC3533 Computación de Alto Rendimiento

Ignacio Labarca – ignacio.labarca@uc.cl

Departamento de Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica de Chile

Jueves 9 de Abril del 2026

Introducción

Modelo de Programación de OpenMP

Directivas OpenMP

Work-sharing

Tareas

Sincronización

Visibilidad de variables

Introducción

Modelo de Programación de OpenMP

Directivas OpenMP

Work-sharing

Tareas

Sincronización

Visibilidad de variables

Código Original

$$\frac{\pi}{4} = \arctan(1) = \int_0^1 \frac{dx}{1+x^2}$$

```
double x, pi, step, sum = 0.0;

step = 1.0 / static_cast<double>(num_steps);

for (int i = 0; i < num_steps; i++) {
   x = (i + 0.5) * step;
   sum += 1.0 / (1.0 + x * x);
}
pi = 4.0 * sum * step;
```

Código con OpenMP

$$\frac{\pi}{4} = \arctan(1) = \int_0^1 \frac{dx}{1+x^2}$$

```
double x, pi, step, sum = 0.0;
step = 1.0 / static_cast<double>(num_steps);

#pragma omp parallel for private(x) reduction(+:sum)
for (int i = 0; i < num_steps; i++) {
   x = (i + 0.5) * step;
   sum += 1.0 / (1.0 + x * x);
}
pi = 4.0 * sum * step;
```

¿Qué es OpenMP?[1]

Open specifications for **Multi-Processing**

- API para usar paralelismo en memoria compartida mediante *multi-threading*.
- Componentes de API
  - Directivas para el compilador.
  - Rutinas de *runtime*.
  - Variables de ambiente.
- Mayores detalles de uso: OpenMP Reference Guide.

[1] https://www.openmp.org/

Lo que NO es OpenMP

- No implementa paralelismo con memoria distribuida.
- No verifica dependencias, conflictos, *race conditions*, *deadlocks*, etc.
- No paraleliza el procesamiento de los datos, sino que paraleliza cómputo.

Lo que SI es OpenMP

- Estándar para múltiples arquitecturas de **memoria compartida**.
- Un conjunto **limitado** de directivas.
- Permite escalar incrementalmente.
- Permite granularizar el paralelismo.
- API para C/C++, Fortran.
- Disponible para Unix/Linux y Windows.

Introducción

Modelo de Programación de OpenMP

Directivas OpenMP

Work-sharing

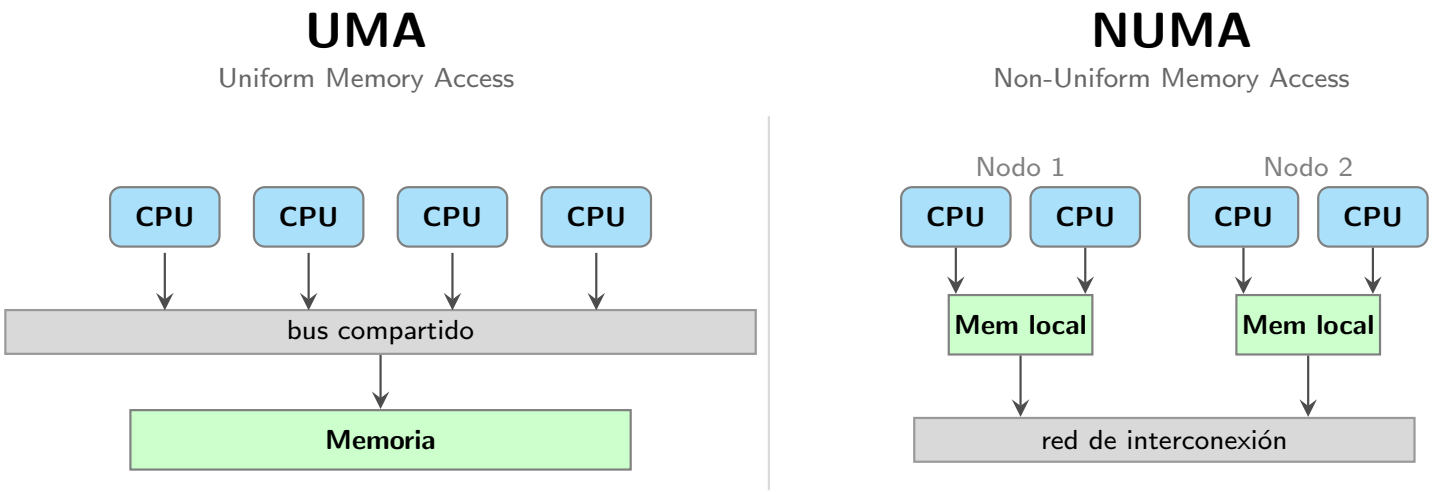
Tareas

Sincronización

Visibilidad de variables

Modelo de memoria compartida

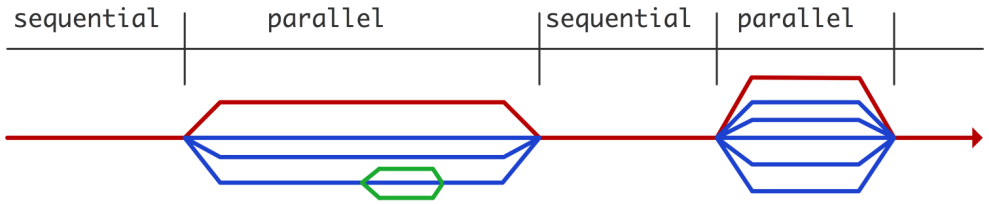
UMA vs NUMA



Thread-based Parallelism

- OpenMP provee paralelismo mediante *threads*.
- Idealmente el número de *threads* debería coincidir con la cantidad de *cores* que puede usar el sistema.
- La paralelización es **explícita**, no automática.
  - Insertar directivas de compilación.
  - Insertar subrutinas con múltiples niveles de paralelismo.

Modelo *Fork-Join*



- *Master-Thread* ejecuta secuencialmente.
- *Fork* al inicio de una región paralela.
- *Join* al final de una región paralela.

Modelo de OpenMP

- Directivas de compilador insertadas en el código fuente.
- Paralelismo anidado: regiones paralelas dentro de otras.
- *Threads* se crean en *runtime*.
- No incluye input/output paralelo.
- Consistencia relajada y memoria temporal en los *threads*.

Introducción

Modelo de Programación de OpenMP

Directivas OpenMP

Work-sharing

Tareas

Sincronización

Visibilidad de variables

Región Paralela

El uso de `parallel` para definir una región paralela.

```
#include <omp.h>

...

#pragma omp parallel
{

   ...

   // Esto se ejecuta en paralelo //

   ...

}
```

Región Paralela

El uso de `parallel` para definir una región paralela.

```
1   #include <iostream>
2   #include <omp.h>
3
4   int main() {
5      int nthreads, tid;
6      #pragma omp parallel private(tid)
7      {
8         // Esto se ejecuta en paralelo //
9
10        tid = omp_get_thread_num();
11        std::cout << "Hello world! I am thread " << tid <<
             "\n";
12
13
14     }
15     return 0;
16  }
```

Región Paralela

El uso de `parallel` para definir una región paralela.

```
1   #include <iostream>
2   #include <omp.h>
3
4   int main() {
5      int nthreads, tid;
6      #pragma omp parallel private(tid)
7      {
8         // Esto se ejecuta en paralelo //
9         tid = omp_get_thread_num();
10        std::cout << "Hello world! I am thread " << tid <<
             "\n";
11
12        #pragma omp barrier
13        if (tid == 0) {
14           nthreads = omp_get_num_threads();
15           std::cout << "Number of threads = " << nthreads <<
                "\n";
16        }
17     }
18     return 0;
```

Hello World

Para compilar nuestro archivo `hello.cpp` hacemos

```
g++ hello.c -fopenmp
```

Podemos usar la variable de ambiente `OMP_NUM_THREADS`:

```
export OMP_NUM_THREADS=4
```

Introducción

Modelo de Programación de OpenMP

Directivas OpenMP

Work-sharing

Tareas

Sincronización

Visibilidad de variables

Work-sharing

Cómo dividir la ejecución del código en cada *thread* que llega hasta ahí.

- No genera nuevos *threads*.
- No hay barrera implícita al entrar, pero sí al salir.

Algunas restricciones

- Debe encontrarse dentro de una región `parallel`.
- Debe ser ejecutado por **todos** los *threads*, o por **ninguno**.
- Dos secciones *work-sharing* consecutivas deben ser vistas por los *threads* **en el mismo orden**.

Work-sharing

El trabajo puede ser dividido de acuerdo a

- *Data parallelism*. Dividir iteraciones de un loop en el *thread-team*

  ```
  #pragma omp for
  ```

- *Functional parallelism.* Divide el trabajo en secciones, cada una ejecutada por un *thread*.

  ```
  #pragma omp sections
  ```

- Serializar el código.

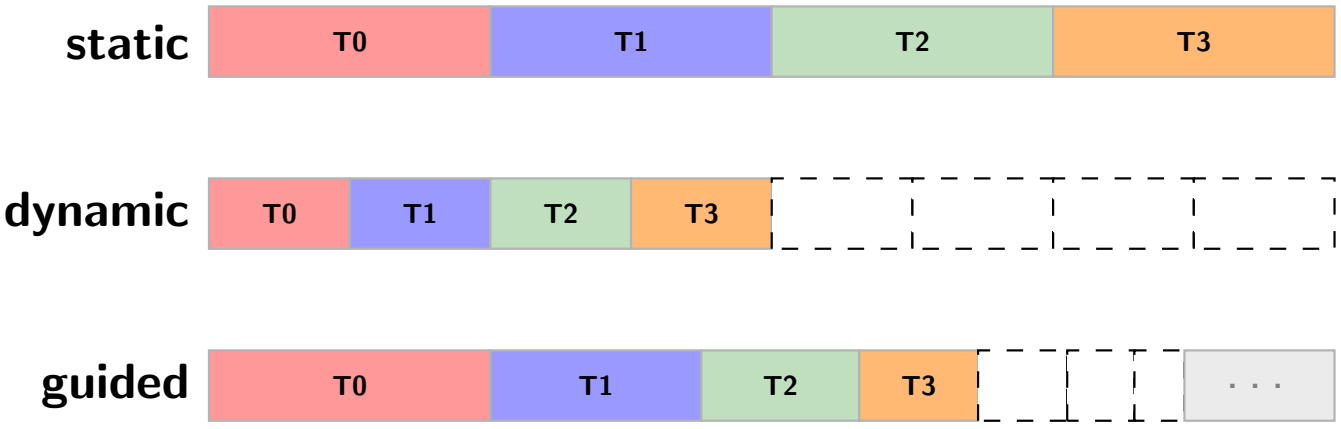
  ```
  # pragma omp single
  ```

#pragma omp for

Con un `schedule` de cómo se dividen las iteraciones:

- `static`. Divide en partes de tamaño `chunk`. No cambia durante la ejecución. Ideal para cargas equilibradas.
- `dynamic`. Divide en partes de tamaño `chunk` y continua asignando dinámicamente. Ideal para iteraciones con distinta carga.
- `guided`. Similar a `dynamic`, pero con un `chunk` $\approx \frac{\text{pendientes}}{\text{procesadores}}$ que decrece.
- `runtime`. Depende de la variable de ambiente `OMP_SCHEDULE`
- `auto`. Deja que el compilador decida.

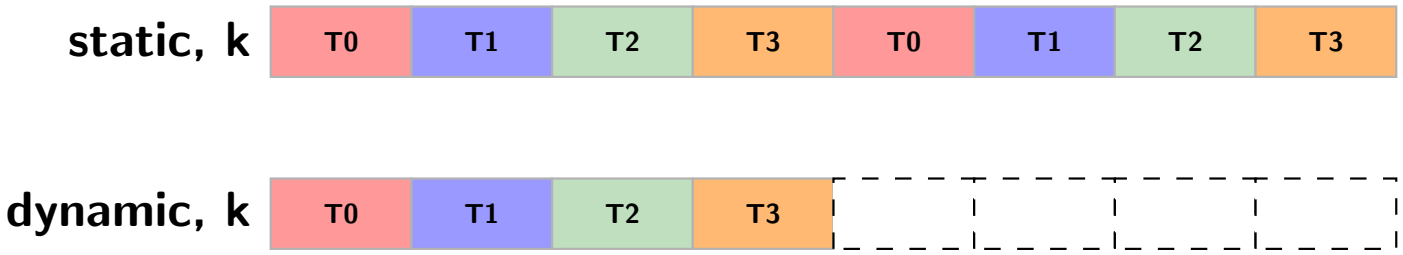
Distribución de iteraciones por `schedule`



Chunk fijo: `schedule(static, k)` y `schedule(dynamic, k)`

Con un `chunk` fijo de tamaño k, las iteraciones se agrupan en bloques de k.

- `static, k`: bloques de tamaño k asignados en *round-robin*. Determinista.
- `dynamic, k`: bloques de tamaño k asignados al primer *thread* libre.



Granularidad

El `chunksize` controla la **granularidad**: el tamaño de cada unidad de trabajo asignada a un *thread*.

- Granularidad fina (chunk pequeño): mejor balance de carga, pero mayor *overhead* de sincronización.
- Granularidad gruesa (chunk grande): menor *overhead*, pero puede generar desbalance.
- El valor óptimo depende del problema: costo por iteración, número de *threads* y variabilidad de la carga.

#pragma omp for

- `nowait`. No hay sincronización al final del loop.
- `ordered`. Iteraciones se ejecutan como en un programa serial.
- `collapse`. Permite que iteraciones de un loop anidado se colapsen en un loop externo antes de dividir.

#pragma omp for

Algunas restricciones:

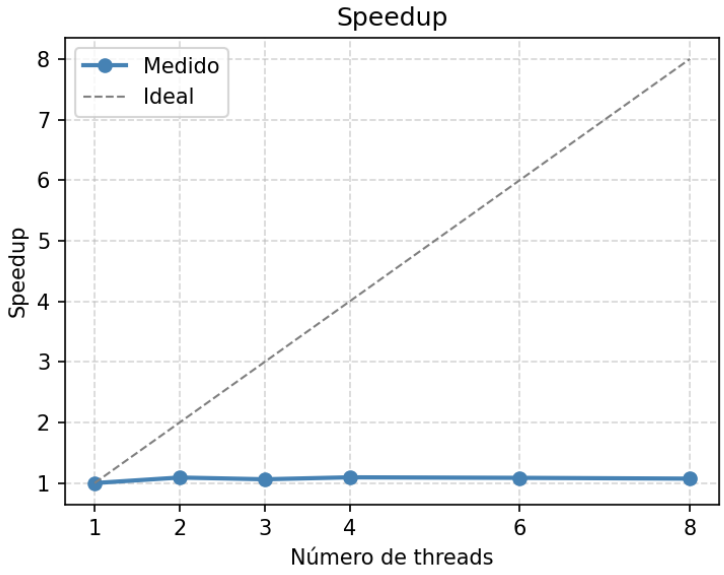
- Debe haber variable de control en el loop.
- Variable de iteración debe ser un tipo `int` y ser común a todos los *threads*.
- `chunk` size debe ser un entero invariante. No se evalúa dinámicamente.

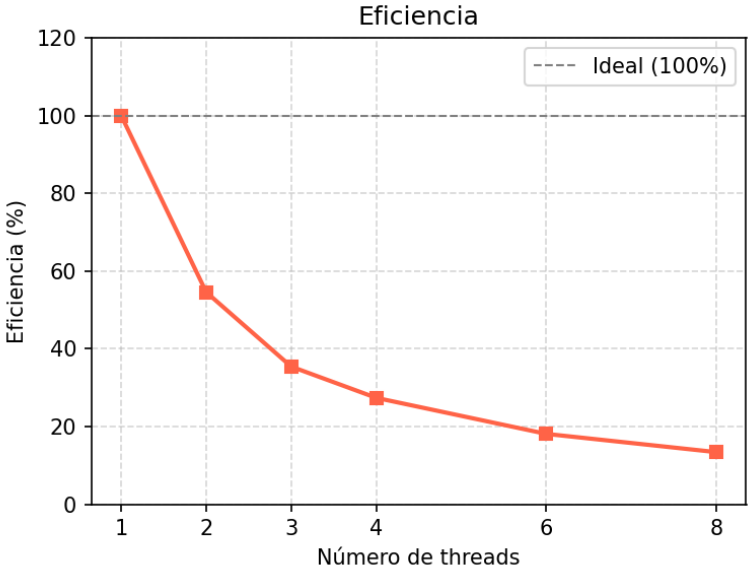
Ejemplo: #pragma omp for

Suma elemento a elemento: `c[i] = a[i] + b[i]`

```
const long N = 100000000;
std::vector<double> a(N), b(N), c(N);
// ... inicializacion ...

#pragma omp parallel for
for (long i = 0; i < N; i++)
    c[i] = a[i] + b[i];
```





Memory-bound vs Compute-bound

La **intensidad aritmética** mide cuántas operaciones se hacen por byte leído de memoria.

- `c[i] = a[i] + b[i]`: 1 suma / 3 accesos a memoria.

  El cuello de botella es el ancho de banda de memoria, no la CPU.

  Agregar threads no ayuda si el bus ya está saturado.

- Para aprovechar el paralelismo se necesita que la operación sea **compute-bound**: mucho cómputo por dato leído.

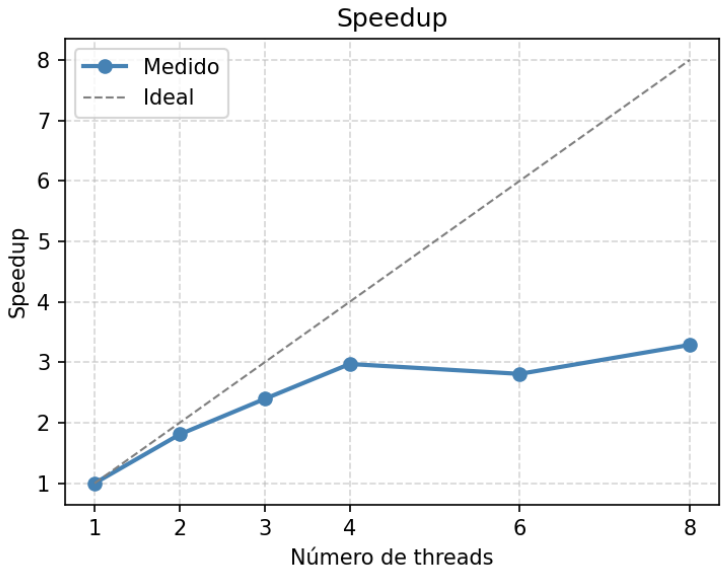
- `c[i] = sin(a[i]) + cos(b[i])`:
  2 funciones trigonométricas / 3 accesos.

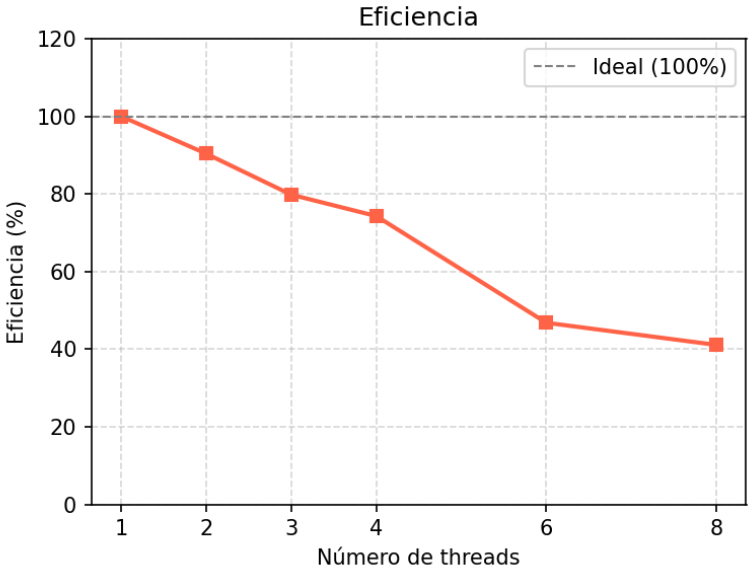
  La CPU es el cuello de botella $\Rightarrow$ más threads ayudan.

Ejemplo: operaciones costosas

Reemplazamos la suma por `sin` y `cos`:

```
1  #pragma omp parallel for
2  for (long i = 0; i < N; i++)
3      c[i] = std::sin(a[i]) + std::cos(b[i]);
```



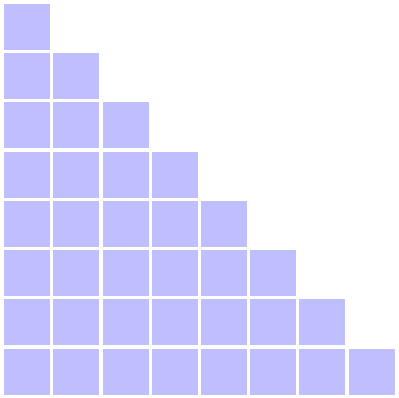


Ejemplo: suma de filas de una matriz triangular

Dada una matriz triangular inferior $A \in \mathbb{R}^{N \times N}$, calcular la suma de cada fila:

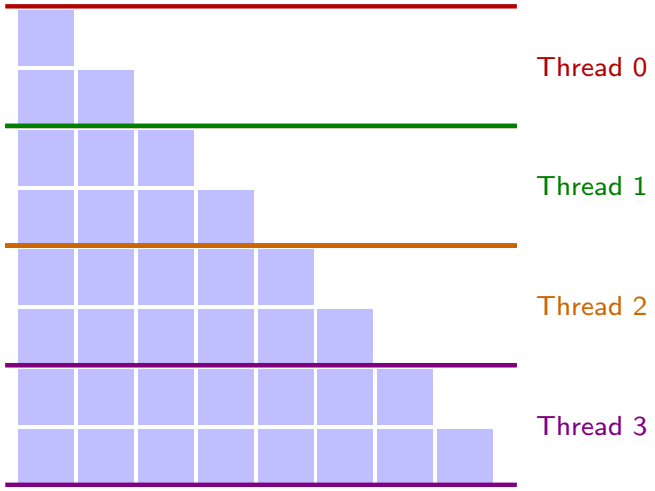
$$r_i = \sum_{j=0}^{i} A_{ij}, \quad i = 0, \ldots, N-1$$

- La fila i tiene $i+1$ elementos.
- La fila 0 tiene 1 elemento; la fila $N-1$ tiene N elementos.
- El trabajo por iteración **crece linealmente** con i.
- Trabajo total: $\dfrac{N(N+1)}{2}$, carga desbalanceada.



Desbalance de carga

Con `schedule(static)`, cada *thread* recibe un bloque contiguo de filas.



El último *thread* acumula las filas más largas: su trabajo es mayor que el del primero.

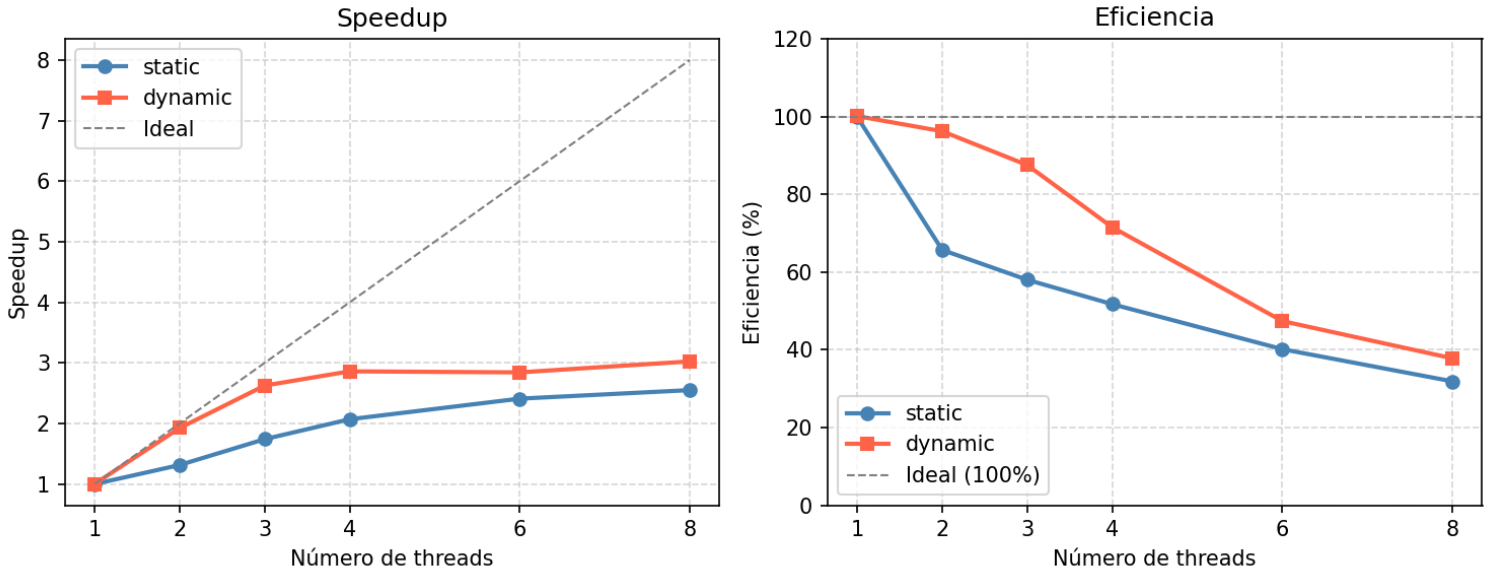
Desbalance de carga: schedule(runtime)

Usando schedule(runtime), el tipo de *schedule* se controla con OMP_SCHEDULE:

```
// fila i tiene i+1 elementos, guardada en offset i*(i+1)/2
#pragma omp parallel for schedule(runtime)
for (long i = 0; i < N; i++) {
    long offset = i * (i + 1) / 2;
    double sum = 0.0;
    for (long j = 0; j <= i; j++)
        sum += data[offset + j];
    result[i] = sum;
}
```

```
OMP_NUM_THREADS=4 OMP_SCHEDULE=static ./triangular_sum
OMP_NUM_THREADS=4 OMP_SCHEDULE=dynamic ./triangular_sum
```

Desbalance de carga: resultados



Work-sharing

Veamos ahora el uso de **secciones**.

```
1   #pragma omp parallel sections
2   {
3      #pragma omp section
4      {
5         printf("Section 1 ejecutada por thread %d\n",
              omp_get_thread_num());
6      }
7      #pragma omp section
8      {
9         printf("Section 2 ejecutada por thread %d\n",
              omp_get_thread_num());
10     }
11     #pragma omp section
12     {
13        printf("Section 3 ejecutada por thread %d\n",
              omp_get_thread_num());
14     }
15  }
```

#pragma omp sections

Código a dividir entre threads.

- Barrera implícita al final (a menos que se use `nowait`).
- Si hay más secciones que *threads*, estos las tomarán a medida que terminen.
- No es posible forzar un orden.
- No puede haber secciones huérfanas (`section` fuera de un bloque `sections`).

Introducción

Modelo de Programación de OpenMP

Directivas OpenMP

Work-sharing

Tareas

Sincronización

Visibilidad de variables

#pragma omp task

Define una tarea independiente que puede ser ejecutada en paralelo.

- Se utiliza dentro de una región `parallel` o `taskgroup`.
- Ideal para trabajo recursivo o tareas con costos variables, como recorrer estructuras de árbol.
- No se ejecuta inmediatamente; se encola para ejecución futura.

Usar `task` dentro de un `for` es equivalente a `schedule(dynamic, 1)`: cada iteración se convierte en una tarea tomada por el primer *thread* disponible.

Tareas con task

```
1   #pragma omp parallel
2   {
3       #pragma omp single
4       {
5           #pragma omp task
6           printf("Tarea 1 por thread %d\n",
                omp_get_thread_num());
7
8           #pragma omp task
9           printf("Tarea 2 por thread %d\n",
                omp_get_thread_num());
10      }
11  }
```

#pragma omp taskwait

Espera a que todas las tareas hijas del *thread* actual terminen.

- Útil para sincronizar subtareas antes de continuar.
- No espera tareas creadas por otros *threads*.

Esperando tareas con taskwait

```
1   #pragma omp parallel
2   {
3       #pragma omp single
4       {
5           #pragma omp task
6           printf("Tarea A\n");
7
8           #pragma omp task
9           printf("Tarea B\n");
10
11          #pragma omp taskwait
12
13          printf("Tareas A y B terminadas\n");
14      }
15  }
```

#pragma omp taskgroup

Agrupa tareas para sincronizar su ejecución.

- Todas las tareas creadas dentro del bloque deben finalizar antes de continuar.
- Ideal para coordinar la finalización de tareas sin múltiples `taskwait`.
- Simplifica la sincronización en bloques de código paralelo.

- Caso de uso: dividir y conquistar, árboles de recursión.

#pragma omp taskgroup

```
#pragma omp parallel
{
   #pragma omp single
   {
      #pragma omp taskgroup
      {
         #pragma omp task
         // Tarea A

         #pragma omp task
         // Tarea B
      }
      // Aqui ya terminaron Tarea A y Tarea B
      // Postproceso
   }
}
```

Aplicación: `merge-sort`

El algoritmo `merge-sort` es del tipo dividir y conquistar.

- Iniciamos con un arreglo de largo N que queremos ordenar.
- La etapa de dividir consiste en dividir el arreglo en dos mitades.
- Ahora queremos ordenar cada una de estas mitades: **paralelizable**.
- Precaución con crear millones de *tasks*: la **granularidad** importa.
  - Umbral de profundidad.
  - Umbral de tamaño.

Introducción

Modelo de Programación de OpenMP

Directivas OpenMP

Work-sharing

Tareas

Sincronización

Visibilidad de variables

#pragma omp master

Ejecuta el bloque de código solo en el *master-thread* (thread 0).

- No implica una barrera implícita.
- Útil para tareas de administración o entrada/salida.

Uso de master

```
1   #pragma omp parallel
2   {
3       #pragma omp master
4       {
5           printf("Solo el master-thread ejecuta esto\n");
6       }
7   }
```

#pragma omp critical

Garantiza exclusión mutua: solo un *thread* entra al bloque a la vez.

- Evita condiciones de carrera.
- Se puede nombrar: `critical(name)`.
- Reduce paralelismo si se abusa.

Reducción con critical

```
int total = 0;

#pragma omp parallel for
for (int i = 0; i < N; i++) {
   int temp = compute(i);
   #pragma omp critical
   {
      total += temp;
   }
}
```

Asegura exclusión mutua, pero puede causar cuellos de botella si se abusa.

#pragma omp barrier

Todos los *threads* deben llegar a este punto antes de continuar.

- Sincronización explícita.
- Puede causar esperas si hay *threads* más lentos.

Sincronización con barrier

```
1   #pragma omp parallel
2   {
3      printf("Antes de la barrera: %d\n",
           omp_get_thread_num());
4
5      #pragma omp barrier
6
7      printf("Despues de la barrera: %d\n",
           omp_get_thread_num());
8   }
```

#pragma omp atomic

Garantiza que una operación simple se realice de forma atómica.

- Más eficiente que `critical` para operaciones simples.
- Sólo se permite una instrucción sobre una variable compartida.
- Evita condiciones de carrera.
- Permite al procesador aprovechar *pipelining*: la instrucción atómica se mapea directamente a una instrucción de hardware.

Reducción con atomic

```
int total = 0;

#pragma omp parallel for
for (int i = 0; i < N; i++) {
   int temp = compute(i);
   #pragma omp atomic
   total += temp;
}
```

Más eficiente que `critical` para operaciones simples como sumas.

`critical` vs `atomic`

Ambas directivas aseguran acceso exclusivo a una variable compartida, pero tienen diferencias clave:

Característica	`critical`	`atomic`
Nivel de protección	Bloque completo	Instrucción específica
Sintaxis	Bloque de código	Una sola operación
Costo	Más alto	Más bajo
Flexibilidad	Cualquier código	Solo operaciones simples
Paralelismo	Menor	Mayor
Ejemplo típico	Sección crítica compleja	Suma de acumulador

Usar `atomic` si la operación es soportada; de lo contrario, usar `critical`.

#pragma omp flush

Garantiza consistencia de memoria entre *threads*.

- Forza actualización de variables compartidas.
- Puede usarse con una lista de variables.
- Útil en protocolos de comunicación entre *threads*.

Consistencia de memoria con flush

```
1   int flag = 0;
2
3   #pragma omp parallel sections
4   {
5      #pragma omp section
6      {
7         flag = 1;
8         #pragma omp flush(flag)
9      }
10
11     #pragma omp section
12     {
13        int local_flag = 0;
14        #pragma omp flush(flag)
15        local_flag = flag;
16        printf("flag: %d\n", local_flag);
17     }
18  }
```

`#pragma omp threadprivate`

Permite que una variable global (o estática) tenga una copia independiente por *thread*.

- Similar a variables privadas, pero persiste entre regiones `parallel`.
- Se declara fuera de las funciones con `threadprivate(var)`.
- Útil para mantener estado local por *thread* a través de distintas regiones paralelas.

Variable privada con threadprivate

```
1   int counter; // Variable global
2   #pragma omp threadprivate(counter)
3
4   void f() {
5      #pragma omp parallel
6      {
7         counter = omp_get_thread_num();
8      }
9
10     #pragma omp parallel
11     {
12        printf("Soy el thread %d con counter = %d\n",
13        omp_get_thread_num(), counter);
14     }
15  }
```

Introducción

Modelo de Programación de OpenMP

Directivas OpenMP

Work-sharing

Tareas

Sincronización

Visibilidad de variables

Visibilidad de variables en OpenMP (1/2)

- `private(var)`: Copia por *thread*, sin inicializar.
- `shared(var)`: Variable común entre todos los *threads*.
- `default(shared/private/none)`: Define visibilidad por omisión.
- `firstprivate(var)`: Copia por *thread*, inicializada desde el *master-thread*.

Visibilidad de variables en OpenMP (2/2)

- `lastprivate(var)`: Copia final sobrescribe la variable original.
- `copyin(var)`: Copia valor del *master-thread* a variables `threadprivate`.
- `copyprivate(var)`: Propaga valor desde una región `single`.
- `reduction(op:var)`: Cada *thread* acumula su copia,
  luego se combinan.

Variables de ambiente en OpenMP

Controlan el comportamiento del **runtime** de OpenMP:

- `OMP_NUM_THREADS`: Número de *threads* a usar.
- `OMP_SCHEDULE`: Define el `schedule` por defecto en `for-loops`.
- `OMP_DYNAMIC`: Permite variar el número de *threads* en tiempo de ejecución.
- `OMP_NESTED`: Habilita regiones paralelas anidadas.
- `OMP_PROC_BIND`: Controla afinidad de *threads* a núcleos (ej. `close`, `spread`).
- `OMP_PLACES`: Define en qué núcleos se pueden ubicar los *threads*.
- `OMP_DISPLAY_ENV`: Muestra configuración activa al ejecutar (`TRUE`/`FALSE`).

Ejemplo: `export OMP_NUM_THREADS=4`